LoRA: Low-Rank Adaptation of Large Language Models

Authors: Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen

ABSTRACT

An important paradigm of natural language processing consists of large-scale pre-training on general domain data and adaptation to particular tasks or domains. As we pre-train larger models, full fine-tuning, which retrains all model parameters, becomes less feasible. Using GPT-3 175B as an example - deploying independent instances of fine-tuned models, each with 175B parameters, is prohibitively expensive. We propose Low-Rank Adaptation, or LoRA, which freezes the pre-trained model weights and injects trainable rank decomposition matrices into each layer of the Transformer architecture, greatly reducing the number of trainable parameters for downstream tasks. Compared to fine-tuning GPT-3 175B with Adam, LoRA can reduce the number of trainable parameters by 10,000 times and the GPU memory requirement by 3 times. LoRA performs on-par or better than fine-tuning in model quality on RoBERTa, DeBERTa, GPT-2, and GPT-3, despite having fewer trainable parameters.

1. Introduction & Storage Bottleneck in Scale

Modern NLP relies on pre-training foundational language models and adapting them to specialized downstream tasks. Conventional adaptation updates all parameters (full fine-tuning). For massive architectures like GPT-3 (175 billion parameters), storing and serving distinct checkpoints requires 350 GB of VRAM per fine-tuned instance, creating extreme operational deployment barriers. We draw on intrinsic dimensionality theory, which asserts that over-parametrized neural models reside on a low intrinsic manifold during task adaptation.

2. Mathematical Formulation of Low-Rank Decomposition

For a pre-trained weight matrix W_0 in R^(d x k), we constrain its weight update Delta W by representing it through low-rank decomposition: W = W_0 + Delta W = W_0 + (alpha / r) * (B * A), where B in R^(d x r), A in R^(r x k), with rank r << min(d, k). Matrix A is initialized from a Gaussian distribution N(0, sigma^2) while B is initialized to zero, ensuring Delta W = 0 at the start of adaptation. The scaling factor alpha / r stabilizes optimization across variable rank choices.

3. Algorithmic Advantages & Zero Deployment Latency

LoRA provides three key engineering benefits:
1. Memory Efficiency: Optimizer states for frozen weights W_0 are omitted from memory, reducing VRAM footprint by up to 3x.
2. Modular Checkpoints: Adapter weights (matrices A and B) occupy mere megabytes (e.g., 20 MB vs 350 GB for GPT-3).
3. Zero Inference Latency: In production deployment, we directly merge weights via W_merged = W_0 + (alpha / r) * (B * A), recovering identical inference latency with zero architectural branching overhead during single-task serving.

4. Quantitative Benchmarks Across Language Models

On GLUE benchmarks using RoBERTa-large and DeBERTa-XXL, LoRA matches or exceeds full fine-tuning accuracy while updating only 0.08% of parameters. On GPT-3 175B, LoRA attains 73.4% on WikiSQL and 65.8 BLEU on E2E NLG, surpassing prefix tuning and series adapters.

5. Limitations and Deployment Vulnerabilities

LoRA exhibits specific constraints: Multi-tenant serving of mixed tasks in a single batched inference call incurs computational overhead because dynamically swapping weight matrices W = W_0 + B_i * A_i per batch element requires custom kernel dispatch. Additionally, determining optimal rank r per projection layer (W_q, W_k, W_v, W_o) currently relies on empirical grid search.

6. Future Work: Adaptive Rank Search & Quantized LoRA

Open questions include automated layer-wise rank allocation, theoretical convergence bounds under non-convex objectives, and unifying LoRA with aggressive 4-bit NormalFloat weight quantization (QLoRA).